

CHANGE REQUEST REPORT

CR06: Dashboard Sidebar & Course Management

University College of Jaffna
Department of Interdisciplinary Studies
Information and Communication Technology
SCM (K72C001M12)
NVQ Level 6 Semester - 1

1. Change Request Details

Change Request ID	CR06
Title	Dashboard Sidebar & Course Management
Priority	Not specified in the supplied implementation plan
Difficulty	Not specified in the supplied implementation plan
Change Type	Frontend layout/routing refactor, new frontend module, backend API, and database schema change
Prepared By	C. Lavasujan
Registration Number	24ICT06

2. Business Requirement

The Admin Dashboard should provide sidebar navigation so administrators can switch between management modules. A new Courses module should allow administrators to create and manage course records through the dashboard.

3. Current Behaviour

- DashboardPage.jsx directly renders the student list.
- Student-management logic is coupled to the DashboardPage layout.
- The Admin Dashboard does not provide the specified Students/Courses sidebar navigation structure.
- A dedicated Courses management interface and supporting course API/database structure are not included in the supplied implementation plan.

4. Requested Change

Refactor the Admin Dashboard into a layout with a Dashboard Sidebar and nested module routes, move student-specific logic into a dedicated StudentsView component, and introduce a Courses module backed by course database, route, controller, and API functionality.

- Change /dashboard into a layout route and add nested /dashboard/students and /dashboard/courses routes.
- Create DashboardSidebar.jsx with Students and Courses navigation links and active-state styling.
- Refactor DashboardPage.jsx to render the sidebar, header, and React Router Outlet.
- Move student fetching, search, and table logic into StudentsView.jsx.
- Create CoursesView.jsx with heading, Add Course button, empty state, and course list/table.
- Create CourseFormModal.jsx with Course Code, Course Name, Description, and Credits fields.
- Add form validation and API submission for course creation.
- Add a courses database table and POST/GET course endpoints, controllers, and route registration.
- Connect the frontend course functions to the backend API and verify functionality.

5. Scope

5.1 In Scope

- Admin Dashboard layout refactoring.
- Sidebar navigation for Students and Courses.
- Nested dashboard routing for Students and Courses.
- Extraction of student-specific logic into StudentsView.jsx.
- New Courses management interface.
- Course creation modal and form validation.
- Frontend course API service functions.
- courses database table.
- Course creation and retrieval API endpoints.
- Course controller and route registration in the backend.
- Integration testing of the course form with the backend API.

5.2 Out of Scope / Not Specified

- Detailed course editing and deletion operations are not specified in the supplied implementation plan.
- Authentication and authorization implementation details for the new course endpoints are not specified in the supplied plan.
- Detailed course fields beyond the example fields are not specified.
- Student-side course enrolment behaviour is not part of this change request.
- Any additional database migration strategy beyond adding the courses table is not specified.

6. Functional Requirements

6.1 Dashboard Layout and Routing

ID	Requirement	Details
FR-001	Dashboard Layout	/dashboard shall render a dashboard layout component containing the sidebar and the active module outlet.
FR-002	Students Route	A nested /dashboard/students route shall be available as the default dashboard module.
FR-003	Courses Route	A nested /dashboard/courses route shall be available for the Courses module.
FR-004	Sidebar Navigation	DashboardSidebar.jsx shall provide navigation links to Students and Courses.
FR-005	Active State Styling	The sidebar shall visually indicate the active navigation state.
FR-006	Dashboard Composition	DashboardPage.jsx shall render the DashboardSidebar, dashboard header, and an <Outlet /> for module content.

6.2 Student View Refactoring

ID	Requirement	Details
FR-007	StudentsView Component	Student-specific dashboard logic shall be moved to frontend/src/pages/admin/StudentsView.jsx.
FR-008	Student Data Logic	StudentsView.jsx shall contain the existing student fetching, search, and table responsibilities moved from DashboardPage.jsx.
FR-009	Existing Student Module	The refactor shall preserve the existing student-management behaviour while changing the layout structure.

6.3 Courses Module - Frontend

ID	Requirement	Details
FR-010	CoursesView	frontend/src/pages/admin/CoursesView.jsx shall provide the Courses management interface.
FR-011	Courses Heading	The Courses section shall display an appropriate heading consistent with the Students section.
FR-012	Add Course Action	The Courses section shall provide an Add Course button.
FR-013	Empty State	The Courses section shall display an appropriate empty state when no courses are available.
FR-014	Course List	The Courses section shall display a list/table of available courses.
FR-015	Course Form Modal	frontend/src/components/CourseFormModal.jsx shall provide a modal form for creating a course.
FR-016	Course Code Field	The course form shall provide a Course Code field.
FR-017	Course Name Field	The course form shall provide a Course Name field.
FR-018	Description Field	The course form shall provide a Description field.
FR-019	Credits Field	The course form shall provide a Credits field.
FR-020	Form Validation	The course form shall validate submitted input before sending the create request.
FR-021	Create Course API Call	The form submission shall send a course-creation request to the backend API, for example POST /api/courses.
FR-022	Course API Service	Course-related frontend API functions shall be added to frontend/src/services/api.js or a dedicated courseService.js.

6.4 Courses Module - Backend

ID	Requirement	Details
FR-023	Courses Table	database/schema.sql shall define a courses table to store course data required by the module.
FR-024	Create Course Endpoint	backend/src/routes/courseRoutes.js shall implement POST /api/courses.
FR-025	List Courses Endpoint	backend/src/routes/courseRoutes.js shall implement GET /api/courses.
FR-026	Course Controller	backend/src/controllers/courseController.js shall contain logic for course creation and retrieval.
FR-027	Route Registration	backend/src/app.js shall register the new course routes.
FR-028	Database Interaction	The course controller shall interact with the database for course creation and retrieval.

6.5 Integration and Verification

ID	Requirement	Details
FR-029	API Integration	The frontend course interface shall be connected to the backend course API.
FR-030	Functional Verification	The completed course form and course list shall be verified for expected create/retrieve behaviour.
FR-031	Layout Verification	The dashboard sidebar, nested routes, and module outlet shall work together without preventing access to the Students module.

7. Security / Access Considerations

The supplied implementation plan identifies the Courses interface as an administrator feature, but it does not specify the authentication or authorization mechanism for the new backend endpoints. Implementation should therefore preserve the application's existing Admin Dashboard access model, while any new endpoint authorization details should be explicitly documented during implementation.

8. Configuration / Potentially Affected Components

8.1 Frontend

- frontend/src/routes/AppRoutes.jsx
- frontend/src/components/DashboardSidebar.jsx
- frontend/src/components/dashboard-sidebar.css
- frontend/src/pages/DashboardPage.jsx
- frontend/src/pages/admin/StudentsView.jsx
- frontend/src/pages/admin/CoursesView.jsx
- frontend/src/components/CourseFormModal.jsx
- frontend/src/services/api.js or a dedicated courseService.js

8.2 Backend

- backend/src/routes/courseRoutes.js
- backend/src/controllers/courseController.js
- backend/src/app.js

8.3 Database

- database/schema.sql - add the courses table.

8.4 Impact Analysis

The impact analysis below identifies the components, behaviours, data structures, and verification areas affected by CR06. It is based on the supplied implementation plan. The impact is concentrated in the Admin Dashboard frontend structure, the new Courses module, the course API/backend, and the database schema. Existing student-management behaviour is expected to remain functional after the refactor. Any additional file changes introduced during implementation should be documented and justified in the Implementation Report.

Impact Area	Affected Components / Files	Impact	Risk / Consideration	Mitigation / Verification
Dashboard Routing	frontend/src/routes/AppRoutes.jsx; DashboardPage.jsx	High	Incorrect nested routing could make Students or Courses inaccessible.	Verify /dashboard/students as the default module and /dashboard/courses as the Courses module.
Dashboard Layout	DashboardPage.jsx; DashboardSidebar.jsx ; dashboard-sidebar.css	High	Layout refactoring may affect existing dashboard presentation and navigation.	Test the side-by-side layout, sidebar active states, header, and module outlet.
Student Management	DashboardPage.jsx; StudentsView.jsx	High	Moving student fetching, search, and table logic may introduce regression.	Compare existing Students behaviour before and after the refactor and verify search/table functionality.
Courses Frontend	CoursesView.jsx; CourseFormModal.jsx ; api.js/courseService.js	High	Incomplete form handling or API integration could prevent course creation or display.	Validate Course Code, Course Name, Description, and Credits; test valid/invalid submissions and course listing.

Backend API	courseRoutes.js; courseController.js; app.js	High	Incorrect routes, controller logic, or registration could break frontend integration.	Verify POST /api/courses and GET /api/courses independently and then through the frontend.
Database	database/schema.sql	High	An incomplete courses table could prevent persistence or retrieval of course records.	Verify schema creation and successful insert/retrieval of course records used by the module.
Access / Security	Admin Dashboard access model; new course endpoints	Medium	The plan identifies Courses as an administrator feature but does not specify new endpoint authorization details.	Preserve the existing Admin Dashboard access model and document any endpoint authorization implemented.
Testing / Regression	Students module; Courses module; routing; API integration	High	The change spans multiple layers, increasing integration and regression risk.	Perform end-to-end verification of navigation, student management, course creation/listing, and route/API connectivity.
Additional Changes	Any files beyond the identified components	Medium	Unplanned file changes can make the implementation scope unclear.	Identify and justify every additional modified file in the Implementation Report.

Overall Impact: CR06 is a multi-layer change with significant impact on dashboard routing and course-management functionality. The highest regression risks are the student-view refactor, nested routing, and frontend-to-backend course integration. These areas should receive priority during verification.

9. Constraints / Assumptions

- The Admin Dashboard remains the entry point for the Students and Courses management modules.
- Students shall remain accessible after the dashboard layout refactor.
- The default nested dashboard route shall be /dashboard/students.
- The Courses module shall be available at /dashboard/courses.
- Course creation shall use the course API rather than direct frontend database access.
- The supplied implementation plan permits example course fields but does not define a complete mandatory schema beyond the named fields.
- Backend and database changes are prerequisites for full frontend course functionality.
- The plan does not specify detailed edit/delete course requirements; these should not be assumed as acceptance criteria.
- Any additional modified files shall be identified and justified in the Implementation Report.
- The existing student-management functionality should continue to work after refactoring.

10. Acceptance Criteria

ID	Requirement	Details
AC-001	Dashboard Layout	The /dashboard route renders the dashboard layout with the sidebar and active module outlet.
AC-002	Students Navigation	The Students module is accessible through /dashboard/students and remains the default dashboard module.
AC-003	Courses Navigation	The Courses module is accessible through /dashboard/courses.
AC-004	Sidebar Active State	The sidebar clearly indicates the currently selected Students or Courses module.
AC-005	Student Refactor	Student fetching, search, and table behaviour works from StudentsView.jsx after the refactor.
AC-006	Courses View	CoursesView.jsx displays the heading, Add Course action, empty state, and course list/table.
AC-007	Course Modal	CourseFormModal.jsx opens as a modal and contains Course Code, Course Name, Description, and Credits fields.
AC-008	Form Validation	Invalid course form input is prevented from being submitted.
AC-009	Course Creation	A valid course submission creates a course through POST /api/courses.
AC-010	Course Retrieval	GET /api/courses returns course data that can be displayed in the Courses view.
AC-011	Database Support	The database schema contains the required courses table for the module.
AC-012	Backend Registration	The new course routes are registered by the backend application.
AC-013	Course Controller	Course creation and retrieval are handled by courseController.js.
AC-014	Frontend API Service	The frontend uses the configured API service function(s) for course operations.

ID	Requirement	Details
AC-015	Existing Students Behaviour	The dashboard refactor does not break the existing Students management module.
AC-016	Integration	The Courses frontend successfully communicates with the backend course API.
AC-017	Additional Changes Documented	Any additional files modified beyond the identified components are documented and justified in the implementation report.

11. Expected Outcome

- The Admin Dashboard provides sidebar navigation between Students and Courses.
- The dashboard uses nested routes with /dashboard/students as the default module and /dashboard/courses for course management.
- DashboardPage.jsx acts as the dashboard layout and module outlet rather than directly containing the student-management logic.
- StudentsView.jsx contains the existing student fetching, search, and table responsibilities.
- Administrators can open the Courses module and create courses through the provided form modal.
- Course data is stored and retrieved through the new courses database table and backend API.
- The frontend course interface is connected to the backend course routes and controller.
- The existing Students module remains functional after the layout refactor.
- Any further file modifications are documented and justified in the implementation report.

12. Implementation Sequence

1. Implement the backend database changes, routes, and controllers so the frontend has an API to use.
2. Create DashboardSidebar and update DashboardPage styling to support the side-by-side dashboard layout.
3. Refactor student logic into StudentsView and configure nested routing.
4. Build CoursesView and CourseFormModal.
5. Connect the course form and course list to the backend API and verify functionality.

End of Change Request Report